

GUÍA DE LABORATORIO N.º 01

Introducción a Spring Boot, arquitectura y configuración del entorno de desarrollo

Desarrollo Web Integrado (100000ST61)

Semana 1 - Unidad 1: API REST

Elemento	Detalle
Curso	Desarrollo Web Integrado
Modalidad	Presencial
Duración sugerida	3 horas académicas
Herramientas	Visual Studio Code, Java 25, Maven, Spring Boot, navegador web, terminal
Producto de la práctica	Proyecto Spring Boot ejecutable en formato JAR con un endpoint básico de verificación

Esta guía está diseñada para que el estudiante instale y valide el entorno de trabajo, comprenda la estructura de un proyecto Spring Boot y construya una primera aplicación web mínima que pueda ejecutarse desde Visual Studio Code y Maven.

1. Introducción

El curso es de naturaleza práctica y aborda tópicos avanzados de desarrollo web, incluyendo HTML, CSS, JavaScript, Spring Boot y servicios RESTful. Además, el logro general indica que el estudiante desarrolla soluciones web integrales mediante frameworks de front-end y back-end, siguiendo el diseño orientado a objetos y tecnologías web para resolver problemas empresariales.

Para la Semana 1, el cronograma define el tema: "Introducción al curso. Spring Boot: conceptos básicos, arquitectura y estructura de un proyecto Spring Boot. Configuración del entorno de desarrollo". También indica como actividades: presentación de objetivos, metodología y herramientas, definición de requisitos iniciales del proyecto y desarrollo de casos.

Enfoque del laboratorio: La práctica no busca desarrollar todavía una API REST completa. El propósito es preparar correctamente el entorno, comprender la arquitectura de Spring Boot y validar que el estudiante puede crear, ejecutar y empaquetar una aplicación base.

2. Logro de aprendizaje de la práctica

Al finalizar la práctica, el estudiante crea, ejecuta y empaqueta un proyecto Spring Boot básico con Maven y Java 25, identificando su estructura principal, el flujo de arranque de la aplicación y el rol inicial de los controladores REST en una aplicación web backend.

Capacidad específica	Evidencia esperada
Configura el entorno de desarrollo backend	Java, Maven, VS Code y extensiones verificadas desde terminal.
Crea un proyecto Spring Boot moderno	Proyecto Maven con empaquetado JAR y dependencia Spring Web.
Reconoce la estructura del proyecto	Explica el uso de src/main/java, src/main/resources, pom.xml y clase principal.
Ejecuta una aplicación local	Aplicación levantada en http://localhost:8080 .
Valida un endpoint básico	Endpoint GET /api/saludo probado desde navegador.

3. Requisitos previos

Antes de iniciar, el estudiante debe contar con una computadora con acceso a Internet y permisos para instalar software. La práctica puede desarrollarse en Windows, macOS o Linux. En laboratorio institucional, el docente puede entregar el proyecto base si las herramientas ya están instaladas.

Software	Uso en la práctica	Validación
Java 25 JDK	Compilar y ejecutar el proyecto Spring Boot.	java -version
Maven	Gestionar dependencias y ejecutar el ciclo de build.	mvn -version
Visual Studio Code	Editor principal del proyecto.	Abrir carpeta del proyecto.
Extension Pack for Java	Soporte Java, Maven, testing y ejecución.	Extensión instalada en VS Code.
Spring Boot Extension Pack	Soporte para proyectos Spring Boot.	Extensión instalada en VS Code.
Navegador web	Probar endpoint GET.	Abrir http://localhost:8080.

Recomendación técnica: En Spring Initializr seleccione siempre una versión estable, no SNAPSHOT ni MILESTONE. Para esta guía se trabaja con Java 25, Maven y empaquetado JAR, de acuerdo con el enfoque del curso.

4. Fundamento teórico necesario

4.1. ¿Qué es Spring Boot?

Spring Boot es un framework que simplifica la creación de aplicaciones Java empresariales y servicios web. Su principal ventaja es que reduce configuración repetitiva, incorpora un servidor embebido y permite ejecutar aplicaciones como archivos JAR.

En cursos de desarrollo web integrado, Spring Boot se utiliza normalmente para construir el backend: es decir, la parte del sistema que expone endpoints, procesa reglas de negocio, se comunica con bases de datos y devuelve respuestas al frontend.

Concepto	Explicación didáctica
Backend	Parte de la aplicación que procesa datos, reglas de negocio, seguridad y persistencia.
Endpoint	URL expuesta por el backend para responder una solicitud HTTP.
API REST	Conjunto de endpoints que permiten comunicación entre sistemas usando HTTP y datos como JSON.
JAR	Archivo ejecutable de Java que empaqueta clases, dependencias y configuración.

4.2. Arquitectura básica de una aplicación Spring Boot

Aunque en la Semana 1 se construirá una aplicación mínima, desde el inicio es importante reconocer la arquitectura general que se usará durante el curso:

Arquitectura conceptual

Cliente HTTP / Navegador / Angular

↓

Controller REST

↓

Service

↓

Repository

↓

Base de datos

En esta primera semana se trabajará principalmente con la clase principal de Spring Boot y un controlador sencillo. Las capas Service, Repository, base de datos, seguridad y frontend serán abordadas progresivamente en semanas posteriores.

Capa	Responsabilidad	Semana de mayor profundidad
Controller	Recibir solicitudes HTTP y devolver respuestas.	Semanas 1 y 2
Service	Organizar lógica de negocio.	Semanas 2 y 3
Repository	Acceder a datos.	Semanas 6 y 7
Security	Autenticación y autorización.	Semanas 8 y 9
Frontend Angular	Consumir la API REST.	Semanas 11 a 16

5. Parte A - Verificación del entorno

La configuración del entorno es crítica. Si Java, Maven o VS Code no están correctamente preparados, los errores aparecerán más adelante y serán difíciles de diagnosticar. Por ello, esta guía inicia validando las herramientas una por una.

5.1. Verificar Java 25

Abra la terminal integrada de Visual Studio Code o una terminal del sistema operativo y ejecute:

```
Terminal
java -version
```

Debe mostrarse una versión de Java 25 o una versión compatible configurada por el docente. Ejemplo esperado:

```
Salida esperada
java version "25"
Java(TM) SE Runtime Environment ...
Java HotSpot(TM) 64-Bit Server VM ...
```

Error frecuente: Si aparece "java no se reconoce como comando interno o externo", Java no está instalado o la variable PATH no está configurada.

5.2. Verificar Maven

```
Terminal
mvn -version
```

Debe mostrarse la versión de Apache Maven y la ruta del JDK que está usando. Es importante verificar que Maven esté usando el mismo Java que se utilizará en el proyecto.

Buena práctica: No basta con que Java funcione. Maven también debe apuntar a un JDK válido, porque será la herramienta encargada de compilar y empaquetar el proyecto.

5.3. Verificar Visual Studio Code

Abra VS Code y confirme que estén instaladas las siguientes extensiones:

- ☐ Extension Pack for Java
- ☐ Spring Boot Extension Pack
- ☐ Maven for Java, incluida normalmente en el Extension Pack for Java
- ☐ REST Client, opcional para probar endpoints desde VS Code

6. Parte B - Creación del proyecto Spring Boot

En esta sección se creará el proyecto base que servirá como punto de partida para las semanas siguientes del curso.

6.1. Ingresar a Spring Initializr

Desde el navegador, ingrese a Spring Initializr. Configure el proyecto con los siguientes valores:

Campo	Valor recomendado
Project	Maven
Language	Java
Spring Boot	Última versión estable disponible, evitando SNAPSHOT o MILESTONE
Group	com.utp
Artifact	semanal
Name	semanal
Description	Proyecto base de Spring Boot para Desarrollo Web Integrado
Package name	com.utp.semanal
Packaging	Jar
Java	25, si está disponible en la versión seleccionada

6.2. Agregar dependencias iniciales

Para la Semana 1 se necesita una dependencia principal:

Dependencia	Motivo
Spring Web	Permite construir aplicaciones web y endpoints REST usando Spring MVC.
Spring Boot DevTools, opcional	Facilita reinicios durante el desarrollo.

Importante: No agregue todavía JPA, MySQL, Security o JWT. Esas dependencias se usarán en semanas posteriores. En esta práctica se busca mantener el proyecto simple y comprensible.

6.3. Descargar y abrir el proyecto

Descargue el archivo ZIP generado por Spring Initializr, descomprímalo y abra la carpeta raíz en Visual Studio Code. No abra la carpeta src directamente; debe abrir la carpeta completa donde está el archivo pom.xml.

```
Estructura inicial
semanal/
├── pom.xml
├── src/
└── mvnw / mvnw.cmd
```

7. Parte C - Reconocimiento de la estructura del proyecto

Antes de escribir código, revise la estructura generada. Comprender esta estructura evitará errores de ubicación de clases y paquetes durante todo el curso.

Elemento	Función
pom.xml	Archivo de configuración Maven. Define dependencias, plugins y versión de Java.
src/main/java	Código fuente principal de la aplicación.
src/main/resources	Archivos de configuración, como application.properties.
src/test/java	Pruebas automatizadas. Será importante al trabajar TDD.
Semana1Application.java	Clase principal que inicia Spring Boot.

7.1. Revisar el pom.xml

Abra el archivo pom.xml e identifique tres elementos: parent de Spring Boot, propiedad de versión Java y dependencia Spring Web.

Fragmento referencial de pom.xml

```
<properties>
  <java.version>25</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

Precisión: El archivo generado puede variar según la versión de Spring Boot. No copie este fragmento si su pom.xml ya fue generado correctamente; úselo solo para reconocer sus partes principales.

8. Parte D - Ejecutar la aplicación por primera vez

Desde la terminal integrada de VS Code, ubicado en la carpeta raíz del proyecto, ejecute:

```
Terminal
mvn spring-boot:run
```

Si la aplicación inicia correctamente, debe observar mensajes indicando que Tomcat se inició en el puerto 8080.

```
Salida esperada
Tomcat started on port 8080
Started Senana1Application
```

8.1. Probar desde el navegador

Abra el navegador e ingrese:

Navegador

`http://localhost:8080`

Es normal que aparezca un error 404, porque todavía no existe una ruta definida. Este 404 no significa que la aplicación esté mal; significa que Spring Boot está ejecutándose, pero aún no existe un endpoint para la raíz /.

Interpretación correcta: Si aparece Whitelabel Error Page o un JSON de error 404, el servidor está activo. Si el navegador indica que no puede conectarse, la aplicación no está levantada o el puerto es incorrecto.

9. Parte E - Crear un primer endpoint de verificación

Aunque el estudio detallado de controladores se profundizará en la Semana 2, en esta práctica se creará un endpoint mínimo para comprobar que la aplicación responde solicitudes HTTP.

9.1. Crear paquete controller

Dentro de `src/main/java/com/utp/semanal`, cree un paquete llamado `controller`. La estructura debe quedar así:

Estructura esperada

```
src/main/java/com/utp/semanal
├── SemanalApplication.java
└── controller
    └── SaludoController.java
```

9.2. Crear la clase SaludoController

Java

```
package com.utp.semanal.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class SaludoController {

    @GetMapping("/saludo")
    public String saludar() {
        return "Hola, Spring Boot está funcionando correctamente";
    }
}
```

9.3. Explicación del código

Elemento	Explicación
@RestController	Indica que la clase responderá solicitudes web y devolverá datos directamente en el cuerpo de la respuesta.
@RequestMapping("/api")	Define una ruta base para todos los endpoints de esta clase.
@GetMapping("/saludo")	Define un endpoint HTTP GET accesible en <code>/api/saludo</code> .

```
return "Hola...";
```

Devuelve texto simple al navegador o cliente HTTP.

Concepto clave: En Spring Boot no se crea manualmente el controlador con new. Spring lo detecta, lo instancia y lo administra como un componente de la aplicación.

10. Parte F - Probar el endpoint

Detenga y vuelva a ejecutar el proyecto si VS Code no reinicia automáticamente la aplicación. Luego abra el navegador y escriba exactamente:

Navegador

```
http://localhost:8080/api/saludo
```

Resultado esperado:

Respuesta

```
Hola, Spring Boot está funcionando correctamente
```

Recordatorio: En el navegador no se escribe "GET http://...", Solo se coloca la URL. El navegador realiza una solicitud GET automáticamente.

11. Parte G - Crear un endpoint de información del proyecto

Ahora se creará un endpoint adicional que devuelva información estructurada. Esto permitirá observar cómo Spring Boot convierte objetos Java en respuestas JSON.

11.1. Crear paquete model

Cree un paquete llamado model y dentro una clase ProyectoInfo.java:

Java

```
package com.utp.semanal.model;

public class ProyectoInfo {

    private String curso;
    private String semana;
    private String tecnologia;
    private String estado;

    public ProyectoInfo(String curso, String semana, String tecnologia, String estado) {
        this.curso = curso;
        this.semana = semana;
        this.tecnologia = tecnologia;
        this.estado = estado;
    }

    public String getCurso() {
        return curso;
    }
}
```

```

    }

    public String getSemana() {
        return semana;
    }

    public String getTecnologia() {
        return tecnologia;
    }

    public String getEstado() {
        return estado;
    }
}

```

11.2. Modificar SaludoController

```

Java
package com.utp.semanal.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.utp.semanal.model.ProyectoInfo;

@RestController
@RequestMapping("/api")
public class SaludoController {

    @GetMapping("/saludo")
    public String saludar() {
        return "Hola, Spring Boot está funcionando correctamente";
    }

    @GetMapping("/info")
    public ProyectoInfo obtenerInfo() {
        return new ProyectoInfo(
            "Desarrollo Web Integrado",
            "Semana 1",
            "Spring Boot + Maven + Java 25",
            "Entorno configurado correctamente"
        );
    }
}

```

11.3. Probar la respuesta JSON

Abra en el navegador:

```

Navegador
http://localhost:8080/api/info

```


Debe obtener una respuesta similar a:

```
JSON
{
  "curso": "Desarrollo Web Integrado",
  "semana": "Semana 1",
  "tecnologia": "Spring Boot + Maven + Java 25",
  "estado": "Entorno configurado correctamente"
}
```

Explicación: Spring Boot incluye Jackson en el starter web. Jackson convierte automáticamente el objeto ProyectoInfo en JSON usando sus métodos getter.

12. Parte H - Configurar application.properties

El archivo application.properties se usa para definir parámetros de configuración. En esta primera práctica se configurará el nombre de la aplicación y el puerto.

```
src/main/resources/application.properties
spring.application.name=semanal
server.port=8080
```

Si el puerto 8080 está ocupado, puede cambiarse temporalmente a 8081:

```
application.properties
server.port=8081
```

Buena práctica: En un proyecto real, las configuraciones sensibles no deben escribirse directamente en el repositorio. Más adelante se trabajará con variables de entorno y perfiles.

13. Parte I - Empaquetar el proyecto como JAR

Una aplicación Spring Boot puede empaquetarse como JAR para ejecutarse sin depender del IDE.

```
Terminal
mvn clean package
```

Al finalizar, Maven generará un archivo en la carpeta target. Ejemplo:

```
Salida esperada
target/semanal-0.0.1-SNAPSHOT.jar
```

13.1. Ejecutar el JAR

Terminal

```
java -jar target/semanal-0.0.1-SNAPSHOT.jar
```

Después de ejecutar el JAR, pruebe nuevamente:

Navegador

```
http://localhost:8080/api/saludo
```

```
http://localhost:8080/api/info
```

Concepto clave: El empaquetado JAR es importante porque más adelante permitirá desplegar la aplicación en servidores o contenedores.

14. Errores frecuentes y solución

Error	Causa probable	Solución
java no se reconoce	JDK no instalado o PATH incorrecto.	Instalar JDK y configurar variables de entorno.
mvn no se reconoce	Maven no instalado o PATH incorrecto.	Instalar Maven o usar mvnw.cmd en Windows.
Puerto 8080 ocupado	Otra aplicación usa el mismo puerto.	Cambiar server.port o cerrar el proceso.
404 al abrir /	No existe endpoint para raíz.	Probar /api/saludo.
Clase no detectada por Spring	Clase creada fuera del paquete base.	Ubicarla dentro de com.utp.semanal o subpaquetes.
Error de compilación	Imports incorrectos o llaves faltantes.	Revisar mensajes de Maven y corregir la clase indicada.

15. Actividad guiada para el estudiante

Desarrolle las siguientes actividades y marque cada una al completarla:

- ☐ Verifiqué Java desde terminal con `java -version`.
- ☐ Verifiqué Maven desde terminal con `mvn -version`.
- ☐ Abri el proyecto desde la carpeta raíz, no desde src.
- ☐ Identifiqué el archivo pom.xml.
- ☐ Identifiqué la clase principal SemanalApplication.java.
- ☐ Ejecuté el proyecto con `mvn spring-boot:run`.
- ☐ Probé que localhost:8080 responde.
- ☐ Creé el paquete controller.
- ☐ Creé el endpoint /api/saludo.
- ☐ Creé la clase ProyectoInfo.
- ☐ Probé el endpoint /api/info y observé una respuesta JSON.
- ☐ Empaqueté el proyecto con `mvn clean package`.

- ☐ Ejecuté el JAR desde terminal.

16. Ejercicios complementarios

Ejercicio 1 - Endpoint de versión

Crear un endpoint GET /api/version que devuelva un texto similar a:

Respuesta esperada

Aplicación Semana 1 - Versión 1.0.0

Ejercicio 2 - Endpoint de estado

Crear un endpoint GET /api/estado que devuelva un objeto JSON con los campos aplicacion, activo y mensaje.

JSON esperado

```
{
  "aplicacion": "semanal",
  "activo": true,
  "mensaje": "Backend disponible"
}
```

Ejercicio 3 - Cambio de puerto

Cambiar temporalmente el puerto de la aplicación a 8081, reiniciar el proyecto y probar los endpoints con el nuevo puerto.

